

# A Multi-Model Web-Grounded Chatbot Platform

Methods and Empirical Results for a Search–Scrape–Synthesize Question-Answering Pipeline

Nicu Alexandru Cristian

1 June 2026

## Abstract

This report documents the methods and complete quantitative results of a *model-agnostic* question-answering platform built around a single pipeline, **search** → **scrape** → **synthesize**, whose final synthesis stage can be served by six interchangeable language models: three cloud large language models (LLMs), one from-scratch  $\sim 124$  M-parameter LLaMA-style transformer trained for this project, and two fine-tuned pretrained sequence-to-sequence (seq2seq) models. Three independent studies are reported: (A) pre-training of the from-scratch model, which converges to a best validation cross-entropy of **2.68**; (B) fine-tuning of the two seq2seq backbones, reaching evaluation losses of **2.23** (DistilBART) and **2.40** (FLAN-T5); and (C) an end-to-end benchmark over ten use cases in which **qwen** attains the highest keyword-coverage quality (**0.96**) and **llama4** the lowest latency (**26.6 s**). All figures were read directly from committed training logs, model checkpoints, and the benchmark CSV. We additionally record the principal threats to validity, notably that the committed benchmark covers only the three cloud models.

| Study | Quantity measured                                      | Headline                                |
|-------|--------------------------------------------------------|-----------------------------------------|
| A     | From-scratch pre-training cross-entropy (3 epochs)     | best val. loss <b>2.68</b>              |
| B     | Seq2seq fine-tuning train/eval cross-entropy (1 epoch) | eval <b>2.23</b> / <b>2.40</b>          |
| C     | End-to-end latency, memory, length, quality (10 cases) | <b>qwen 0.96</b> ; <b>llama4 26.6 s</b> |

## Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                         | <b>2</b> |
| <b>2</b> | <b>Methods</b>                              | <b>2</b> |
| 2.1      | Model-agnostic pipeline . . . . .           | 2        |
| 2.2      | Search . . . . .                            | 2        |
| 2.3      | Scraping . . . . .                          | 2        |
| 2.4      | Synthesis . . . . .                         | 3        |
| 2.5      | From-scratch model architecture . . . . .   | 3        |
| 2.6      | From-scratch training methodology . . . . . | 3        |
| 2.7      | Seq2seq fine-tuning methodology . . . . .   | 3        |
| 2.8      | Dataset construction . . . . .              | 4        |
| 2.9      | Evaluation methodology . . . . .            | 4        |

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>3 Results</b>                                       | <b>4</b> |
| 3.1 Study A — From-scratch training dynamics . . . . . | 4        |
| 3.2 Study B — Seq2seq fine-tuning metrics . . . . .    | 5        |
| 3.3 Study C — End-to-end benchmark . . . . .           | 5        |
| <b>4 Threats to Validity</b>                           | <b>5</b> |
| <b>5 Reproducibility</b>                               | <b>6</b> |

## 1 Introduction

Every model in the platform runs an identical three-stage flow; only the *synthesizer* changes, which makes cross-model comparison meaningful. A user question is turned into a web search, the top sources are scraped and cleaned, and the relevant content is synthesized into a sourced answer. The six registered models span three families:

1. **Cloud LLMs** (via Groq / Ollama): `llama4`, `qwen`, `mistral`.
2. **A from-scratch LLaMA-style transformer** (~124 M parameters), trained from random initialization for this project.
3. **Two fine-tuned seq2seq models**: `distilbart-cnn-12-6` and `flan-t5-small`.

A critical reproducibility note is recorded up front: the committed benchmark run (Study C) contains *only* the three cloud models (`llama4`, `qwen`, `mistral`). The from-scratch and fine-tuned models were trained (Studies A and B) but were not included in the committed `evaluation/results.csv`.

## 2 Methods

### 2.1 Model-agnostic pipeline

A FastAPI gateway dispatches each request on `MODELS[model_key].provider: groq/ollama` route to the cloud agent, `local` to the from-scratch agent, and `finetuned` to the seq2seq agent. The `MODELS` registry (`configs/settings.py`) is the single source of truth for all six models, and heavy local weights are imported lazily on first use.

### 2.2 Search

The primary provider is **Tavily** (LLM-oriented snippets, deduplicated URLs), with a no-API-key **DuckDuckGo** fallback. The provider is selected by the `SEARCH_PROVIDER` variable; both are normalized to a common `{title, url, snippet}` shape. The configured cap is `MAX_SEARCH_RESULTS = 10`.

### 2.3 Scraping

The primary extractor is **Trafilatura** (which strips navigation, ads, and boilerplate), with a plain `requests` fallback for sites that block it. Per-URL content is capped at `MAX_CONTENT_LENGTH/N` characters (`MAX_CONTENT_LENGTH = 1500`,  $N$  = number of URLs), favouring *breadth of sources over depth* of any single page.

## 2.4 Synthesis

Three synthesis families are implemented (Table 1). The cloud agent uses a deliberate **map–reduce**: each source is summarized to bullet points (*map*), then a single call combines them (*reduce*), keeping the final prompt short.

**Table 1:** Synthesis families and mechanisms.

| Family       | Module                          | Mechanism                                                                                                                              |
|--------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Cloud        | <code>search_agent.py</code>    | Map–reduce over Groq/Ollama LLMs via LangChain <code>PromptTemplate</code> chains                                                      |
| From-scratch | <code>local_agent.py</code>     | LLaMA-style transformer; tool-formatted input, autoregressive report generation; <code>validate_tool_output()</code> guards each stage |
| Fine-tuned   | <code>finetuned_agent.py</code> | DistilBART or FLAN-T5 conditional generation; falls back to base HF weights if no local fine-tune exists                               |

Registered cloud models: `llama4 = meta-llama/llama-4-scout-17b-16e-instruct` (Groq), `qwen = qwen/qwen3-32b` (Groq), and `mistral = mistral` (local Ollama).

## 2.5 From-scratch model architecture

A decoder-only, LLaMA-style transformer trained from random initialization (Table 2). The design decision that resolved an earlier convergence failure is a set of eight **tool-aware special tokens** that make the training format match the runtime pipeline’s tool output: `<|tool_start|>`, `<|tool_end|>`, `<|search_result|>`, `<|scrape_result|>`, `<|source|>`, `<|url|>`, `<|report_start|>`, and `<|report_end|>`.

**Table 2:** From-scratch transformer architecture.

| Component                            | Choice                                                                              |
|--------------------------------------|-------------------------------------------------------------------------------------|
| Parameters                           | ~124 M                                                                              |
| Layers / heads                       | 12 / 12                                                                             |
| $d_{\text{model}}$ / $d_{\text{ff}}$ | 768 / 2048                                                                          |
| Context window                       | 2048 tokens                                                                         |
| Positional encoding                  | RoPE (rotary, base 10 000)                                                          |
| Normalization                        | RMSNorm (pre-norm), $\epsilon = 10^{-6}$                                            |
| Feed-forward                         | SwiGLU (SiLU-gated)                                                                 |
| Embedding / LM head                  | Weight-tied                                                                         |
| Attention                            | <code>scaled_dot_product_attention</code> , causal                                  |
| Tokenizer                            | GPT-2 BPE (50 257) + 8 special $\rightarrow$ 50 265 vocab                           |
| Initialization                       | $\mathcal{N}(0, 0.02)$ ; residual projections scaled $1/\sqrt{2 n_{\text{layers}}}$ |

## 2.6 From-scratch training methodology

Next-token cross-entropy with `ignore_index = -100`. Crucially, **tool/context tokens are masked** ( $-100$ ); only the report span `<|report_start|>...<|report_end|>` is supervised. Hyperparameters are listed in Table 3. Training data (`combined_all`) mixes CNN/DailyMail, XSum, BillSum ( $\leq 18\,000$ ), SAMSum ( $\leq 14\,000$ ), and a custom agent-grounded set, each rendered into the tool-aware template with **30% multi-source augmentation** (context budget  $\leq 1600$  tokens, report budget  $\leq 400$  tokens).

## 2.7 Seq2seq fine-tuning methodology

Two pretrained backbones were fine-tuned on the same (`article`, `summary`) corpus via the HuggingFace `Seq2SeqTrainer` (Table 4).

**Table 3:** From-scratch training hyperparameters (verified from `best.pt`).

| Setting           | Value                                                 |
|-------------------|-------------------------------------------------------|
| Objective         | next-token cross-entropy, masked context              |
| Optimizer         | AdamW, $\beta = (0.9, 0.95)$ , $\epsilon = 10^{-8}$   |
| Weight decay      | 0.05 (norms & biases excluded)                        |
| Learning rate     | $3 \times 10^{-4}$ peak, cosine decay to $10^{-5}$    |
| Warmup            | 2000 steps                                            |
| Dropout           | 0.1                                                   |
| Batch             | $4 \times 8$ grad-accum = 32 effective                |
| Epochs            | 3                                                     |
| Gradient clipping | max-norm 1.0                                          |
| Mixed precision   | bf16 (CUDA) / fp16 (MPS) / fp32 (CPU)                 |
| Device router     | CUDA > MPS > CPU                                      |
| Validation        | every 500 steps, $\leq 100$ batches; best by val loss |

**Table 4:** Seq2seq fine-tuning configuration.

| Setting              | DistilBART                                | FLAN-T5                         |
|----------------------|-------------------------------------------|---------------------------------|
| Base model           | sshleifer/distilbart-cnn-12-6<br>(~306 M) | google/flan-t5-small (~77–80 M) |
| Task prefix          | none                                      | "summarize: "                   |
| Learning rate        | $5 \times 10^{-5}$                        | $5 \times 10^{-5}$              |
| Batch size           | 8                                         | 8                               |
| Epochs               | 1                                         | 1                               |
| Max input / target   | 512 / 128                                 | 512 / 128                       |
| Eval / save strategy | per epoch                                 | per epoch                       |
| Precision            | fp16 (CUDA)                               | fp16 (CUDA)                     |

## 2.8 Dataset construction

The custom fine-tuning set is *self-generated*: the live search+scrape+summarize pipeline is run over the ten evaluation questions, producing `{instruction, context, response, category}` triples. **Eight of ten** use cases produced valid examples (the *Scientific* and *Ambiguous* categories were dropped by the validation guards), so `training/dataset.json` holds eight examples.

## 2.9 Evaluation methodology

Each model is run over each of ten use cases. Per-run metrics are wall-clock latency (`time_sec`), peak Python allocations via `tracemalloc` (`memory_kb`; process-level, *not* model residency), answer length in characters, a keyword-coverage `quality_score`, and a boolean `success` (the call returned without raising). The quality score is a pure lexical-overlap proxy:

$$\text{score} = \frac{\#\{\text{expected topics found in answer}\}}{\#\{\text{expected topics}\}}, \quad \text{score} \in [0, 1].$$

Robustness uses up to two retries with back-off on 429/RESOURCE\_EXHAUSTED and a 2 s inter-call delay.

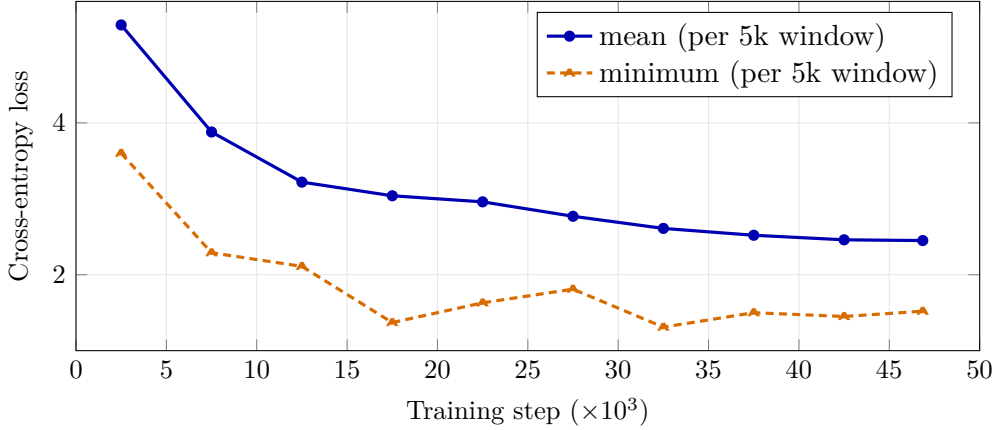
## 3 Results

### 3.1 Study A — From-scratch training dynamics

Source: `training/checkpoints/training_log.json` (954 logged points, every 50 optimizer steps) and the saved `best.pt/latest.pt` checkpoints. Summary statistics are given in Table 5; the smoothed loss trajectory is plotted in Figure 1 and tabulated in Table 6.

**Table 5:** From-scratch training summary.

| Quantity                                                          | Value                              |
|-------------------------------------------------------------------|------------------------------------|
| Logged optimizer steps                                            | 50 $\rightarrow$ 47 700 (3 epochs) |
| Epoch boundaries (step)                                           | 0@50, 1@15 950, 2@31 850           |
| Initial train loss                                                | 9.31                               |
| Trailing-window mean train loss (last 40 logs)                    | $\sim$ 2.51                        |
| Global minimum train loss                                         | 1.31                               |
| <b>Best validation loss</b> ( <code>best.pt</code> , step 47 500) | <b>2.68</b>                        |
| <code>latest.pt</code> (step 47 000) train loss                   | 2.84                               |
| Gradient norm (start / min / end)                                 | 5.40 / 1.31 / 2.92                 |

**Figure 1:** Smoothed from-scratch training loss. The mean trends monotonically to  $\sim$ 2.5, resolving the prior “loss stuck above 3.5” failure; instantaneous minima fall as low as 1.31.

*Interpretation.* The model converges monotonically. The best saved *validation* loss is 2.68; the README prose “past 2.5” refers to the *training*-loss trend.

### 3.2 Study B — Seq2seq fine-tuning metrics

Source: `checkpoint-5626/trainer_state.json` for each model (Table 7).

*Interpretation.* DistilBART reaches a lower eval loss (2.23 vs. 2.40), consistent with its larger capacity and summarization pre-training; FLAN-T5-small is  $\sim 3.5\times$  faster at evaluation and  $\sim 4\times$  cheaper in FLOPs.

### 3.3 Study C — End-to-end benchmark

Source: `evaluation/results.csv` — 30 runs (3 models  $\times$  10 use cases), timestamped 20 April 2026 (Table 8).

*Interpretation.* Closed, factual, and procedural queries score highest; *Scientific*, *Ambiguous*, and *Multi-hop* are hardest — partly genuine difficulty, partly metric artefact (e.g. a correct quantum-computing answer that never uses the expected keyword “google” is penalised).

## 4 Threats to Validity

1. **Quality metric is lexical, not semantic.** Keyword coverage cannot distinguish a sourced claim from a hallucinated one, nor credit a correct paraphrase lacking the exact keyword.

**Table 6:** Smoothed training loss (mean and minimum per 5000-step window).

| Step window     | Mean loss | Min loss |
|-----------------|-----------|----------|
| 0 – 4 999       | 5.29      | 3.60     |
| 5 000 – 9 999   | 3.88      | 2.29     |
| 10 000 – 14 999 | 3.22      | 2.11     |
| 15 000 – 19 999 | 3.04      | 1.37     |
| 20 000 – 24 999 | 2.96      | 1.63     |
| 25 000 – 29 999 | 2.77      | 1.81     |
| 30 000 – 34 999 | 2.61      | 1.31     |
| 35 000 – 39 999 | 2.52      | 1.50     |
| 40 000 – 44 999 | 2.46      | 1.45     |
| 45 000 – 47 700 | 2.45      | 1.52     |

**Table 7:** Seq2seq fine-tuning metrics (1 epoch each).

| Metric                              | DistilBART            | FLAN-T5               |
|-------------------------------------|-----------------------|-----------------------|
| Epochs                              | 1.0                   | 1.0                   |
| Global steps                        | 5 626                 | 5 626                 |
| Train batch size                    | 8                     | 8                     |
| First logged train loss (step 50)   | 2.72                  | 2.74                  |
| Last logged train loss (step 5 600) | 1.73                  | 2.22                  |
| <b>Eval loss (epoch 1)</b>          | <b>2.23</b>           | <b>2.40</b>           |
| Eval runtime                        | 34.2 s                | 9.7 s                 |
| Eval throughput                     | 43.8 smp/s            | 154.8 smp/s           |
| Total FLOPs                         | $3.48 \times 10^{16}$ | $8.37 \times 10^{15}$ |

2. **success**  $\neq$  **correctness**. It only means “no exception.” Two **mistral** runs (use cases 3 and 6) returned raw tool-call JSON rather than prose yet were marked successful (and scored 0.4).
3. **Local / fine-tuned models absent from Study C**. The committed benchmark covers only the three cloud models; cross-family quality comparison is not yet evidenced by the CSV.
4. **Memory figures are not model footprints**. **tracemalloc** captures Python-level allocations only.
5. **Validation breadth (Study A)**. Validation loss is computed over  $\leq 100$  batches every 500 steps; the single best value (2.68) is reported.
6. **Single benchmark run**. No repeated trials, hence no variance or confidence intervals on latency or quality.
7. **Small custom corpus**. The agent-grounded set is eight examples; its contribution is qualitative (format/style grounding), not statistical.

## 5 Reproducibility

Key artefacts and the commands that regenerate each study are listed below (Table 11).

```
# From-scratch training (Study A)
python training/train_scratch.py          # full run
python training/train_scratch.py --test   # smoke test

# Seq2seq fine-tuning (Study B)
```

**Table 8:** End-to-end benchmark, committed run (3 cloud models).

| Model   | n  | Succ. | Avg time      | Time range  | Avg qual.   | Avg mem (KB) <sup>†</sup> |
|---------|----|-------|---------------|-------------|-------------|---------------------------|
| qwen    | 10 | 100%  | 66.9 s        | 7.9–122.7 s | <b>0.96</b> | 5 610                     |
| mistral | 10 | 100%  | 54.8 s        | 7.4–221.5 s | 0.80        | 324                       |
| llama4  | 10 | 100%  | <b>26.6 s</b> | 20.7–33.1 s | 0.70        | 9 378                     |

<sup>†</sup> `tracemalloc` peak Python allocations at the agent layer — *not* model weight residency. Heavy cloud inference is server-side, so these reflect orchestration overhead only.

**Table 9:** Per-model quality distribution (10 use cases each).

| Model   | Quality scores (sorted)                          | # perfect (1.0) |
|---------|--------------------------------------------------|-----------------|
| qwen    | 0.8, 0.8, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0 | 8 / 10          |
| mistral | 0.4, 0.4, 0.6, 0.8, 0.8, 1.0, 1.0, 1.0, 1.0, 1.0 | 5 / 10          |
| llama4  | 0.4, 0.4, 0.4, 0.6, 0.6, 0.8, 0.8, 1.0, 1.0, 1.0 | 3 / 10          |

```
python training/finetune.py --model distilbart --epochs 1
python training/finetune.py --model flan-t5 --epochs 1
```

```
# Benchmark (Study C)
python evaluation/benchmark.py
```

**Key hyperparameters (verified from saved checkpoints).** From-scratch: `lr=3e-4`, `wd=0.05`, `dropout=0.1`, `batch=4x8`, `warmup=2000`, `epochs=3`, `max_seq_len=2048`, `d_model=768`, `n_layers=12`, `n_heads=12`, `d_ff=2048`, `vocab=50257+8`. Seq2seq: `lr=5e-5`, `batch=8`, `epochs=1`, `max_input=512`, `max_target=128`.

All numbers in this report were read directly from committed artefacts (`training_log.json`, `best.pt/latest.pt`, the two `trainer_state.json` files, and `results.csv`) on 1 June 2026.

**Table 10:** Per-category mean quality across all three models.

| Category         | Mean quality ( $n = 3$ ) |
|------------------|--------------------------|
| Factual          | 1.00                     |
| Coding           | 1.00                     |
| Technical How-To | 0.93                     |
| Financial        | 0.93                     |
| Comparative      | 0.80                     |
| Trend Analysis   | 0.80                     |
| Current Events   | 0.73                     |
| Multi-hop        | 0.73                     |
| Ambiguous        | 0.67                     |
| Scientific       | 0.60                     |

**Table 11:** Artefacts underlying each study.

| Artefact                    | Path                                                                                           |
|-----------------------------|------------------------------------------------------------------------------------------------|
| From-scratch model + loop   | training/model.py, training/train_scratch.py                                                   |
| Training log / checkpoints  | training/checkpoints/{training_log.json, best.pt, latest.pt}                                   |
| Seq2seq fine-tuning         | training/finetune.py; states in<br>finetuned_*/_checkpoints/checkpoint-5626/trainer_state.json |
| Dataset builder / data      | training/create_dataset.py, training/dataset.json,<br>training/data_utils.py                   |
| Benchmark harness / results | evaluation/benchmark.py, evaluation/use_cases.py,<br>evaluation/results.csv                    |
| Model registry / config     | configs/settings.py                                                                            |